

Materia przedmiotu

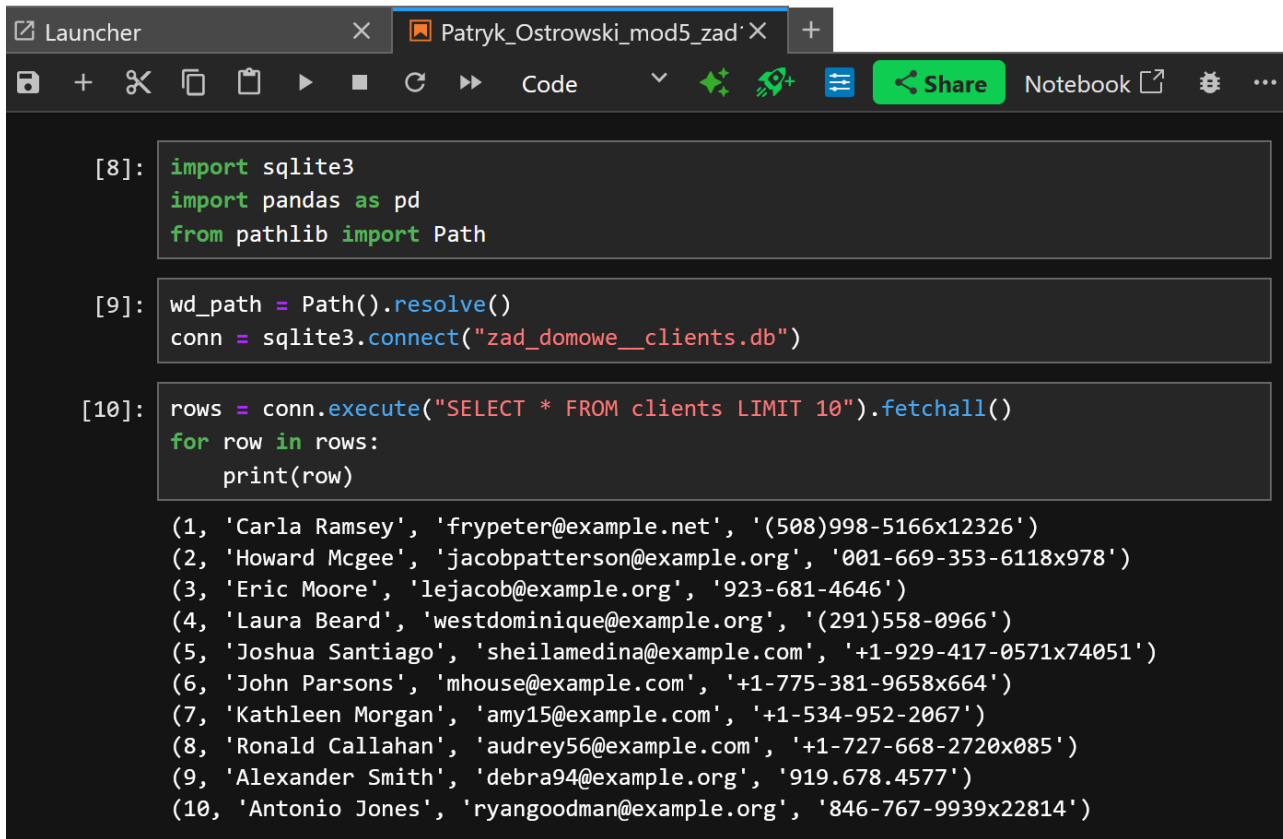
Dostępny jest plik .csv a także tabela relacyjnej bazy danych. Są to struktury przedstawiające wymiary, obie zostają wczytane jako data frame. Nie da się ich jednak połączyć logicznie w sposób odzwierciedlający wiarogodną tabelę faktów. Jedynym na to sposobem jest interpretacja dostarczonych faktur.

Sposób rozwiązania problemu

Dostarczone faktury poddane są interpretacji przez model gpt-4o-mini, w celu uzyskania struktury JSON, a następnie ta struktura wiąże dane z w/w widoków wymiarów – towarów i klientów – w widok odpowiadający tabeli faktów z relacyjnej bazy danych.

Rozwiązanie krok po kroku z użyciem Jupyter Notebook

1. Niektóre importy i ustawienie środowiska, odczyt pierwszych 10-ciu wierszy z tabeli.



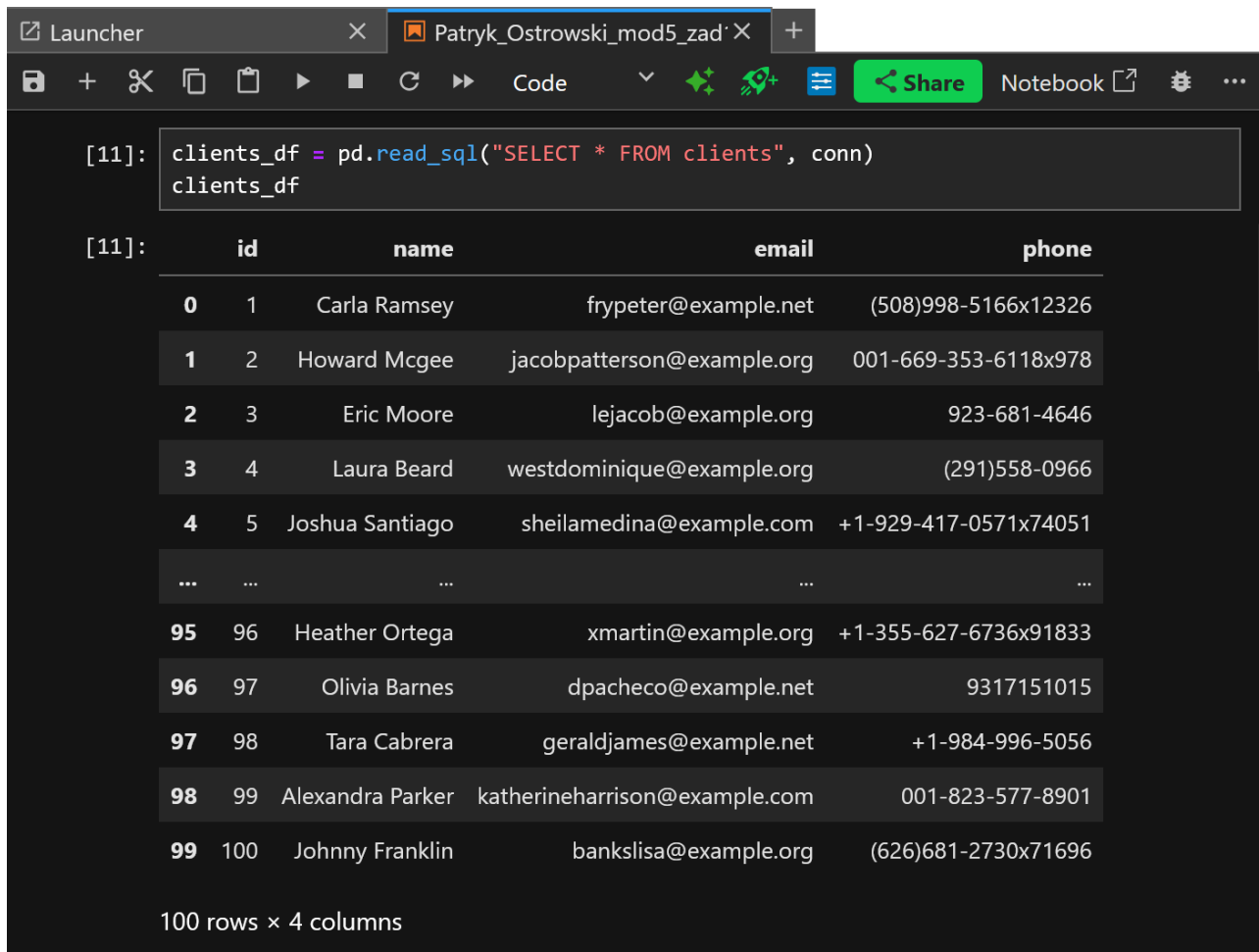
```
[8]: import sqlite3
import pandas as pd
from pathlib import Path

[9]: wd_path = Path().resolve()
conn = sqlite3.connect("zad_domowe__clients.db")

[10]: rows = conn.execute("SELECT * FROM clients LIMIT 10").fetchall()
for row in rows:
    print(row)

(1, 'Carla Ramsey', 'frypeter@example.net', '(508)998-5166x12326')
(2, 'Howard McGee', 'jacobpatterson@example.org', '001-669-353-6118x978')
(3, 'Eric Moore', 'lejacob@example.org', '923-681-4646')
(4, 'Laura Beard', 'westdominique@example.org', '(291)558-0966')
(5, 'Joshua Santiago', 'sheilamedina@example.com', '+1-929-417-0571x74051')
(6, 'John Parsons', 'mhouse@example.com', '+1-775-381-9658x664')
(7, 'Kathleen Morgan', 'amy15@example.com', '+1-534-952-2067')
(8, 'Ronald Callahan', 'audrey56@example.com', '+1-727-668-2720x085')
(9, 'Alexander Smith', 'debra94@example.org', '919.678.4577')
(10, 'Antonio Jones', 'ryangoodman@example.org', '846-767-9939x22814')
```

2. Wczytanie danych z tabeli do data frame:



The screenshot shows a Jupyter Notebook interface with two tabs: 'Launcher' and 'Patryk_Ostrowski_mod5_zad'. The notebook contains two code cells. The first cell executes `clients_df = pd.read_sql("SELECT * FROM clients", conn)`. The second cell displays the resulting DataFrame, which has 100 rows and 4 columns: `id`, `name`, `email`, and `phone`. The output shows a preview of the data, including rows for Carla Ramsey, Howard McGee, Eric Moore, Laura Beard, Joshua Santiago, Heather Ortega, Olivia Barnes, Tara Cabrera, Alexandra Parker, and Johnny Franklin. Below the preview, it states '100 rows x 4 columns'.

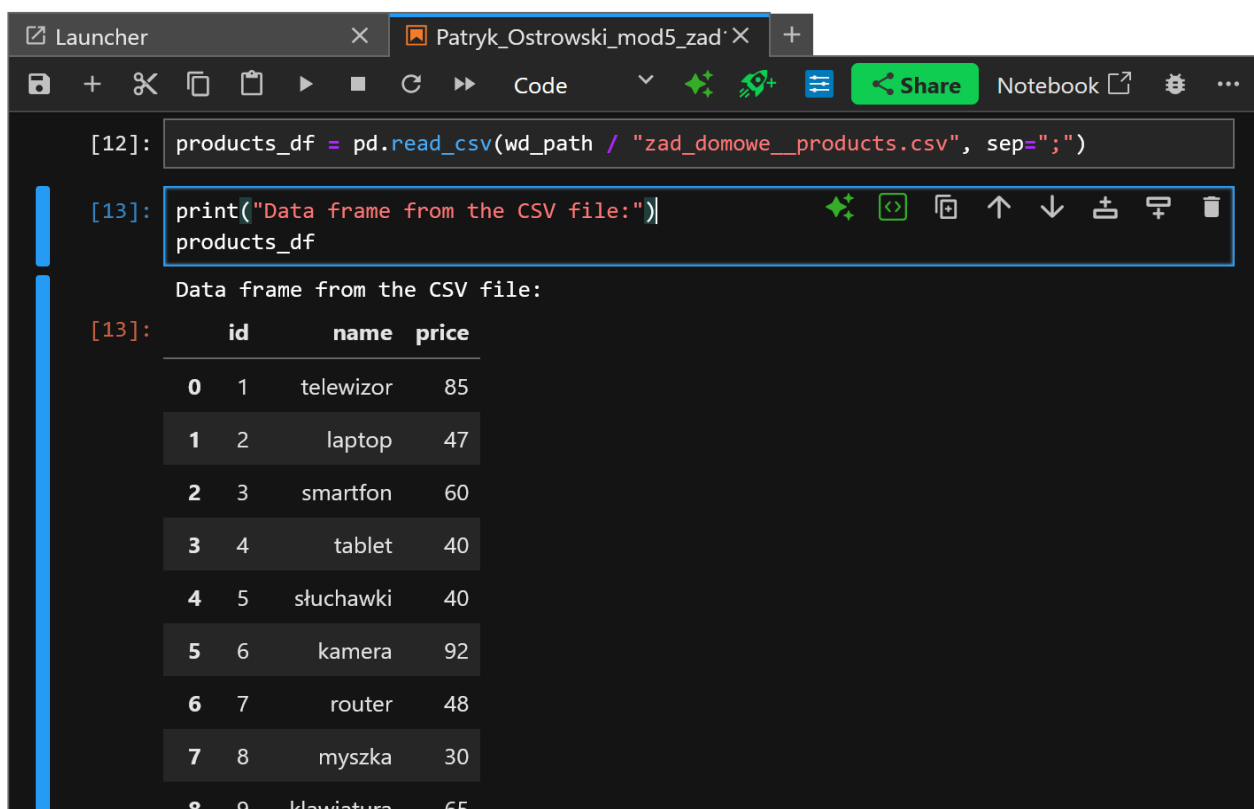
```
[11]: clients_df = pd.read_sql("SELECT * FROM clients", conn)
clients_df
```

```
[11]:
```

	id	name	email	phone
0	1	Carla Ramsey	frypeter@example.net	(508)998-5166x12326
1	2	Howard McGee	jacobpatterson@example.org	001-669-353-6118x978
2	3	Eric Moore	lejacob@example.org	923-681-4646
3	4	Laura Beard	westdominique@example.org	(291)558-0966
4	5	Joshua Santiago	sheilamedina@example.com	+1-929-417-0571x74051
...	...	...	...	...
95	96	Heather Ortega	xmartin@example.org	+1-355-627-6736x91833
96	97	Olivia Barnes	dpacheco@example.net	9317151015
97	98	Tara Cabrera	geraldjames@example.net	+1-984-996-5056
98	99	Alexandra Parker	katherineharrison@example.com	001-823-577-8901
99	100	Johnny Franklin	banksalisa@example.org	(626)681-2730x71696

100 rows x 4 columns

3. Wczytanie danych z pliku .csv do data frame:



The screenshot shows a Jupyter Notebook interface with two tabs: 'Launcher' and 'Patryk_Ostrowski_mod5_zad'. The notebook contains two code cells. The first cell executes `products_df = pd.read_csv(wd_path / "zad_domowe_products.csv", sep=";")`. The second cell executes `print("Data frame from the CSV file:")` followed by `products_df`. The output shows the text 'Data frame from the CSV file:' followed by a preview of the DataFrame, which has 9 rows and 4 columns: `id`, `name`, and `price`. The preview shows rows for 'telewizor', 'laptop', 'smartfon', 'tablet', 'słuchawki', 'kamera', 'router', and 'myszka'.

```
[12]: products_df = pd.read_csv(wd_path / "zad_domowe_products.csv", sep=";")
```

```
[13]: print("Data frame from the CSV file:")
products_df
```

Data frame from the CSV file:

```
[13]:
```

	id	name	price
0	1	telewizor	85
1	2	laptop	47
2	3	smartfon	60
3	4	tablet	40
4	5	słuchawki	40
5	6	kamera	92
6	7	router	48
7	8	myszka	30
8	9	klawiatura	65

4. Uwierzytelnienie użytkownika kluczem OpenAI w celu późniejszego przetwarzania danych:

```
Launcher Patryk_Ostrowski_mod5_zad'X +
+ ✂ 📄 ▶ ■ ↺ ▶▶ Code ▾ ✨ 🚀 📄 < Share Notebook 📄 ...

[14]: from getpass import getpass
      from openai import OpenAI

      openai_key = getpass("Enter your OpenAI key:")
      openai_client = OpenAI(api_key=openai_key)

      Enter your OpenAI key: .....
```

5. Import kolejnych bibliotek i utworzenie klas wraz z ich atrybutami:

```
Launcher Patryk_Ostrowski_mod5_zad'X +
+ ✂ 📄 ▶ ■ ↺ ▶▶ Code ▾ ✨ 🚀 📄 < Share Notebook 📄 ...

[15]: from pydantic import BaseModel
      from datetime import date

      class InvoiceInfoItem(BaseModel):
          description: str
          product_id: int
          quantity: int
          price: float

      class InvoiceInfo(BaseModel):
          company_name: str
          customer_id: int
          customer_name: str
          invoice_number: int
          date: date
          items: list[InvoiceInfoItem]

[16]: import base64
      import instructor

      instructor_openai_client = instructor.from_openai(OpenAI(api_key=openai_key))
      orders = []
```

6. Iteracja po katalogu w celu pobrania plików z określoną częścią wspólną nazwy i konkretnym rozszerzeniem, a następnie określenie docelowej struktury odczytanych danych:

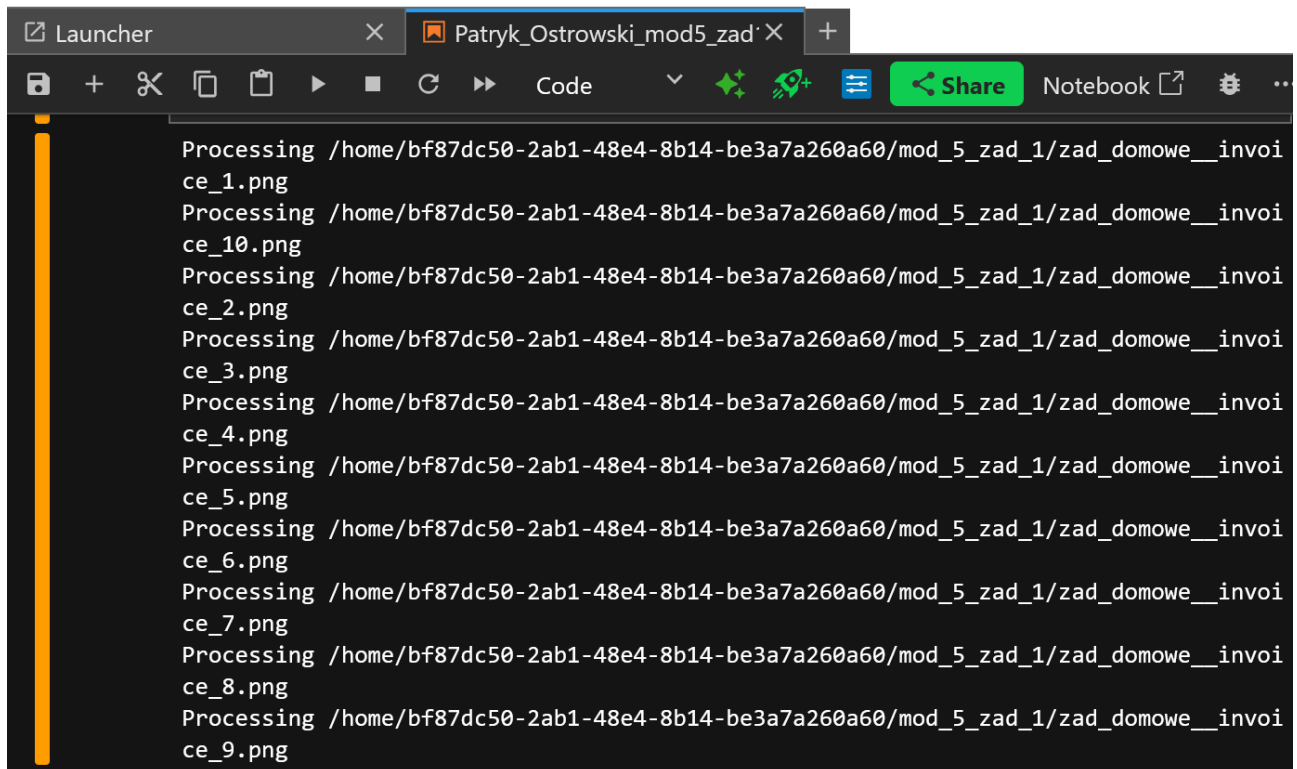
```
Launcher Patryk_Ostrowski_mod5_zad' X +
+ ✂ 📄 ▶ ■ ↺ ⏩ Code ⌵ 🌟 🚀 📄 < Share Notebook 📄 ⚙️ ...

•[17]: for image_path in sorted(wd_path.rglob("zad_domowe__invoice_*.png")):
    print(f"Processing {image_path}")
    with open(image_path, "rb") as f:
        image_data = base64.b64encode(f.read()).decode('utf-8')

    invoice_info = instructor_openai_client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {
                "role": "system",
                "content": "Jesteś pomocnym asystentem, który potrafi odczytywać dane z faktur",
            },
            {
                "role": "user",
                "content": [
                    {
                        "type": "text",
                        "text": "Odczytaj dane z tej faktury i zwróć je jako strukturę JSON",
                    },
                    {
                        "type": "image_url",
                        "image_url": {
                            "url": f"data:image/png;base64,{image_data}"
                        },
                    },
                ],
            },
        ],
        response_model=InvoiceInfo,
        max_tokens=2000,
    )
```

```
invoice_data = invoice_info.model_dump()
for item in invoice_data["items"]:
    order = {
        "company_name": invoice_data["company_name"],
        "customer_id": invoice_data["customer_id"],
        "customer_name": invoice_data["customer_name"],
        "invoice_number": invoice_data["invoice_number"],
        "date": invoice_data["date"],
        "description": item["description"],
        "product_id": item["product_id"],
        "quantity": item["quantity"],
        "price": item["price"],
    }
    orders.append(order)
```

7. Przetwarzanie obrazów wg podanej instrukcji:



The screenshot shows the Anaconda Jupyter Notebook interface. The top bar includes a 'Launcher' button, a tab for 'Patryk_Ostrowski_mod5_zad', and a '+'. Below the top bar is a toolbar with icons for saving, adding, deleting, copying, pasting, running, and other notebook functions. The main area displays a series of 'Processing' commands for image files. A vertical yellow bar is visible on the left side of the notebook area.

```
Processing /home/bf87dc50-2ab1-48e4-8b14-be3a7a260a60/mod_5_zad_1/zad_domowe__invoi
ce_1.png
Processing /home/bf87dc50-2ab1-48e4-8b14-be3a7a260a60/mod_5_zad_1/zad_domowe__invoi
ce_10.png
Processing /home/bf87dc50-2ab1-48e4-8b14-be3a7a260a60/mod_5_zad_1/zad_domowe__invoi
ce_2.png
Processing /home/bf87dc50-2ab1-48e4-8b14-be3a7a260a60/mod_5_zad_1/zad_domowe__invoi
ce_3.png
Processing /home/bf87dc50-2ab1-48e4-8b14-be3a7a260a60/mod_5_zad_1/zad_domowe__invoi
ce_4.png
Processing /home/bf87dc50-2ab1-48e4-8b14-be3a7a260a60/mod_5_zad_1/zad_domowe__invoi
ce_5.png
Processing /home/bf87dc50-2ab1-48e4-8b14-be3a7a260a60/mod_5_zad_1/zad_domowe__invoi
ce_6.png
Processing /home/bf87dc50-2ab1-48e4-8b14-be3a7a260a60/mod_5_zad_1/zad_domowe__invoi
ce_7.png
Processing /home/bf87dc50-2ab1-48e4-8b14-be3a7a260a60/mod_5_zad_1/zad_domowe__invoi
ce_8.png
Processing /home/bf87dc50-2ab1-48e4-8b14-be3a7a260a60/mod_5_zad_1/zad_domowe__invoi
ce_9.png
```

* - w komórce tej pojawia się żółta belka informująca o nowowprowadzonych zmianach, z którymi program nie był uruchamiany. Zmiana ta polegała na usunięciu zduplikowanego importu biblioteki pathlib – zduplikowanego przy przeklejaniu fragmentów kodu z IDE VSCode do edytora anacondy. Program wykonał się poprawnie co widać na kolejnych screenach.

8. Utworzenie finalnego data frame, łączącego wymiary w sposób dający pełny wgląd w to, kto co kupił i kiedy:

```
Launcher Patryk_Ostrowski_mod5_zad1 X +
```

```
[18]: orders_df = pd.DataFrame(orders)

print("Orders data frame:")
orders_df
```

Orders data frame:

[18]:	company_name	customer_id	customer_name	invoice_number	date	description
0	Firma XYZ	3	Eric Moore	2	2024-05-25	router
1	Firma XYZ	3	Eric Moore	2	2024-05-25	narty
2	Firma XYZ	92	Valerie Stevens	11	2024-05-02	sluchawki
3	Firma XYZ	60	Troy Alexander	3	2024-05-19	ochraniacze
4	Firma XYZ	78	William Velazquez	4	2024-03-26	rolki (id produktu 14)
5	Firma XYZ	33	Steven Jackson	5	2024-07-23	pendrive (id produktu 10)
6	Firma XYZ	33	Steven Jackson	5	2024-07-23	laptop (id produktu 2)
7	Firma XYZ	8	Ronald Callahan	6	2024-03-14	sluchawki (id produktu 5)
8	Firma XYZ	8	Ronald Callahan	6	2024-03-14	ochraniacze (id produktu 16)
9	Firma XYZ	4	Laura Beard	7	2024-06-24	telewizor
10	Firma XYZ	63	Alexis Johnson	8	2024-02-21	klawiatura
11	Firma XYZ	63	Alexis Johnson	8	2024-02-21	router

9. Sprawdzenie manualne.

Odczytana kodem Pythona tabela pokazuje po pięć pierwszych i ostatnich wierszy:

```
Data frame from the table:
```

	id	name	email	phone
0	1	Carla Ramsey	frypeter@example.net	(508)998-5166x12326
1	2	Howard McGee	jacobpatterson@example.org	001-669-353-6118x978
2	3	Eric Moore	lejacob@example.org	923-681-4646
3	4	Laura Beard	westdominique@example.org	(291)558-0966
4	5	Joshua Santiago	sheilamedina@example.com	+1-929-417-0571x74051
..	...	...	...	...
95	96	Heather Ortega	xmartin@example.org	+1-355-627-6736x91833
96	97	Olivia Barnes	dpachecc@example.net	9317151015
97	98	Tara Cabrera	geraldjames@example.net	+1-984-996-5056
98	99	Alexandra Parker	katherineharrison@example.com	001-823-577-8901
99	100	Johnny Franklin	banksalisa@example.org	(626)681-2730x71696

10. Odczytany kodem Pythona plik .csv pokazujący wszystkie towary:

```
Data frame from the csv file:
```

	id	name	price
0	1	telewizor	85
1	2	laptop	47
2	3	smartfon	60
3	4	tablet	40
4	5	słuchawki	40
5	6	kamera	92
6	7	router	48
7	8	myszka	30
8	9	klawiatura	65
9	10	pendrive	47
10	11	rower	69
11	12	hulajnoga	44
12	13	skateboard	81
13	14	rolki	11
14	15	kask	46
15	16	ochraniacze	46
16	17	narty	55
17	18	snowboard	85
18	19	kurtka	12
19	20	spodnie	14

Eric Moore miał kupić router i narty – zgadza się:

Klient: Eric Moore (id klienta 3)

Faktura #: 2

Data: 2024-05-25

Opis	Ilość	Cena
router (id produktu 7)	3	\$48.00
narty (id produktu 17)	5	\$55.00

Laura Beard miała kupić telewizor – zgadza się:

Klient: Laura Beard (id klienta 4)

Faktura #: 7

Data: 2024-06-24

Opis	Ilość	Cena
telewizor (id produktu 1)	2	\$85.00

Johny Franklin miał kupić spodnie – zgadza się:

Klient: Johnny Franklin (id klienta 100)

Faktura #: 9

Data: 2024-01-26

Opis	Ilość	Cena
spodnie (id produktu 20)	2	\$14.00

Powyższe screeny są do zweryfikowania w danych umieszczonych na dwóch poprzedzających stronach – punkty 8., 9. i 10.

Podsumowanie i wnioski

Ćwiczenie bardzo pouczające. Wstępne dane, czyli zbiory danych w postaci pliku .csv i tabela relacyjnej bazy danych, z którą należało się połączyć, odzwierciedlają realia projektowe i po kilku-kilkunastu ćwiczeniach własnych tego samego w ramach poprzedniej części kursu wydają się dziecinnie proste, bo jednocześnie w pracy inżyniera danych z pewnością stanowią chleb powszedni.

Część zadania polegająca na wysłaniu do modelu i interpretacji obrazów także była świetnym powtórzeniem lekcji nt., tym bardziej, że część kodu musiała zostać dodana – tu oczywiście z pomocą przychodzą narzędzia zewnętrzne. Kod jednak nie jest przytłaczający, a powracanie do niego pomaga utrwalać przebieg czynności programistycznych.

Na koniec przeprowadzona kontrola dla 3 z 15-tu transakcji pokazuje, że przy tym poziomie powagi zadania nie ma potrzeby dalszego sprawdzania spójności danych – 20-procentowe pokrycie testem manualnym wydaje się wystarczające.